

Caller-side precondition checking, and Eval_and_throw

Abstract

This paper describes how and why to do caller-side(-only) precondition checking and explains how that makes a violation handling mode where exceptions are thrown on a contract violation viable.

The general problem with a throwing violation handling mode is its interaction with noexcept functions. But if preconditions are checked at the call site, they can be checked before a function body is entered, and thus they can avoid termination when a contract violation throws, and then it's fine to use such a throwing violation handling mode even with noexcept functions.

More rationale

Caller-side(-only) precondition checking is useful for other things than just making Eval_and_throw work. It has two major benefits:

1. It matches some ostensibly common uses well; it's probably often so that programmers are interested in whether the functions their code calls are called correctly, and it's less important to find bugs in the functions called, so preconditions of called code are of higher interest.
2. It allows binary libraries to ship just one version; there's no need to ship a separate contract-enabled version for checking preconditions, and a contract-disabled version for performance. Callers can just turn on precondition checking at the caller side, and can then verify that calls are correct, without having to enable precondition checks at the definition side. This seems *_very_* attractive from the perspective of a library vendor, and also from the perspective of a library user.

Design goals

The most major goal here is to provide the ability to do caller-side precondition checking without any cooperation from the callee. No extra symbols, no other symbol table tricks, no shared tables between translation units. No ABI break when enabling/disabling contract checking in the callee, or in the caller.

I think it's important that the Contracts design allows the above regardless of what the contents of the MVP are. In particular, I consider it massively important that it's possible to implement the contracts MVP without any ABI impact, in other words, I consider it massively important that it's possible to implement the MVP so that enabling or disabling contracts doesn't change ABI. Regardless of whether this proposal ever gets adopted.

Target ship vehicle

I am not suggesting that this proposal should be concerned for the current Contracts MVP, or in C++26. However, if we end up entertaining Eval_and_throw for C++26, I think we need to entertain this paper as well.

So how does it work?

This is relatively simple; when the compiler sees a call to an overload set, i.e. a call that is not performed via a pointer-to-function or a pointer-to-member-function, it can check the preconditions of the call target after overload-resolving it, and change the function call to be an expression that performs the precondition check and then calls the function.

In pseudo-code, a call

`f()`

is transformed into

```
((precond() ? nop() : violation()), f())
```

where `precond()` is a function that evaluates the precondition and returns the result of that evaluation, `nop()` is dummy function that does nothing and returns void, and `violation()` is a call to a violation handler, which either aborts in the case of the MVP's `Eval_and_abort`, or throws with the suggested `Eval_and_throw`.

This is doable in a relative straightforward fashion in just a compiler front-end.

What about indirect calls?

For pointers-to-function, this could be done so that a call to a function actually calls a thunk, so when the address of a function is taken, the address of the thunk is used instead. But we might want to avoid mandating that, because..

..it becomes seriously difficult to handle indirect member function calls. If the function is virtual, we'll have an index into a vtable element, and we can't just thunk it. We would need to add additional vtable slots, and that seems like a seriously awkward thing to require.

For now, I'm suggesting that if we end up entertaining this sort of checking mode, we plainly state that it doesn't work for calls through pointers to functions or pointers to member functions.

So how does this make `Eval_and_throw` work?

Well, as we saw, a call

`f()`

is transformed into

```
((precond() ? nop() : violation()), f())
```

In this code, if `violation()` throws, we won't hit a possible `noexcept` of `f()`, because we never call `f()`. So the throwing precondition check works even if the target function is `noexcept`.

So what is being proposed here?

I am proposing that we, eventually, not necessarily as part of the MVP, add a contract-checking mode that

1. enables precondition checking of functions called by the code in the current TU, and

2. enables that checking for overloaded calls only, so calls to functions and member functions that name the function called, and
3. enables checking of contract asserts in the current TU, and
4. does not enable checking of any postconditions, even in the current TU (I suppose this is debatable for the current TU), and
5. offers no guarantees on whether the preconditions are evaluated once or more often.

I am proposing that regardless of whether we adopt an `Eval_or_throw`. In case we do adopt that option as a violation handling mode, I propose that enabling that mode also implicitly enables exactly what is enabled in the bullet list above, and nothing more.

What is the performance impact of this proposal?

You may end up evaluating preconditions twice. But in general, a precondition check is in an inline function, and such a function may be codegen-emitted twice, both in the calling and in the called TU. If it's all in a single TU, it's all visible to an optimizer. And even in multiple different TUs, it's visible to link-time optimization. The duplicate calls can be eliminated if the optimizer can prove them to be side-effect free.